

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY



(SUBJECT: Artificial Neural Network s)
(SUBJECT CODE:24SBT113)

A MINI PROJECT REPORT ON
“Credit Card Fraud Detection.”

Submitted in the partial fulfillment of the requirements for the Fourth semester

BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence & Machine Learning

Submitted By,

Pavan S Poojary	01SU24AI071
Prathvi Prashanth Naik	01SU24AI076
Vignesh S Poojary	01SU24AI116
Prateek	01SU24AI074
Vishal Shetty	01SU24AI128

UNDER THE GUIDANCE OF

Prof. Mahesh Kumar V B
Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146

2025-26

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

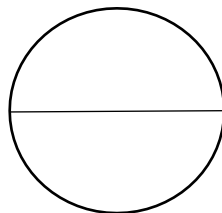


Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that, **Pavan S Poojary (01SU24AI071), Prathvi Prashanth Naik (01SU24AI076), Vignesh S Poojary (01SU24AI116), Prateek (01SU24AI074), Vishal Shetty (01SU24AI118)** has satisfactorily completed the assessment (Mini Project) in **Artificial Neural Networks (24SAL113)** prescribed by the Srinivas University for the 4th semester B. Tech course during the year **2025-26**.

MARKS AWARDED



Staff In charge

Name: Prof. Mahesh Kumar V B

TABLE OF CONTENTS

Sl.no	Description	Page no.
1	Abstract	1
2	Introduction	2
3	Objectives of the Project	3
4	Literature Review	5
5	Dataset Description	7
6	System Architecture	11
7	Methodology	15
8	Machine Learning and Deep Learning Models	24
9	Evaluation Metrics and Results	30
10	Applications, Advantages and Limitations	36
11	Conclusion and Future Work	39
12	References	41
13	Appendix	42

Abstract

Credit card fraud has become a significant challenge in the modern financial industry due to the rapid growth of online transactions and digital payment systems. Fraudulent activities lead to substantial financial losses for banks, businesses, and customers. Therefore, developing an efficient and reliable fraud detection system is essential to ensure the security of financial transactions. Traditional fraud detection methods based on predefined rules are often unable to detect new and complex fraud patterns. As a result, machine learning and deep learning techniques have gained popularity for detecting fraudulent transactions.

This project focuses on the development of a **Credit Card Fraud Detection System using Machine Learning (ML), Deep Learning (DL), and Artificial Neural Networks (ANN)**. The system analyzes historical credit card transaction data to identify patterns that distinguish fraudulent transactions from legitimate ones. The dataset used in this project contains multiple features such as transaction time, transaction amount, and several anonymized variables obtained through Principal Component Analysis (PCA).

One of the major challenges in fraud detection is the **class imbalance problem**, where fraudulent transactions represent only a small percentage of the total dataset. To address this issue, the **Synthetic Minority Oversampling Technique (SMOTE)** is applied to balance the dataset and improve the ability of the models to detect fraudulent activities. Several models including **Random Forest, XGBoost, and Artificial Neural Networks** are implemented and trained to classify transactions.

The performance of these models is evaluated using various evaluation metrics such as **accuracy, precision, recall, F1-score, confusion matrix, and ROC curve**. The results show that machine learning and deep learning models can effectively detect fraudulent transactions with high accuracy while minimizing false positives.

Overall, the proposed system demonstrates the effectiveness of intelligent data-driven techniques in identifying credit card fraud and enhancing financial security. This project highlights the importance of advanced analytical methods in preventing fraudulent activities and protecting digital financial transactions.

1. Introduction

The rapid advancement of digital technology has significantly transformed the financial sector, especially with the widespread use of credit cards and online payment systems. Credit cards provide a convenient and secure way for customers to perform transactions without carrying physical cash. However, the increasing popularity of electronic payments has also led to a rise in credit card fraud. Fraudsters use stolen card information or unauthorized access to perform illegal transactions, resulting in significant financial losses for banks, businesses, and customers.

Credit card fraud detection has therefore become an important area of research in the fields of data mining, machine learning, and cybersecurity. Financial institutions process millions of transactions daily, making it difficult to manually monitor and detect fraudulent activities. Traditional fraud detection systems rely on rule-based methods that use predefined patterns or thresholds to identify suspicious transactions. Although these systems can detect known fraud patterns, they often fail to identify new or complex fraud techniques.

With the advancement of artificial intelligence, machine learning and deep learning techniques have emerged as powerful tools for fraud detection. These techniques analyze historical transaction data and automatically learn patterns that distinguish legitimate transactions from fraudulent ones. Machine learning algorithms can process large datasets efficiently and continuously improve their performance as more data becomes available.

One of the major challenges in credit card fraud detection is the imbalanced nature of the dataset. In most cases, fraudulent transactions represent only a very small percentage of the total number of transactions. This imbalance makes it difficult for machine learning models to accurately identify fraud cases because the models may become biased toward the majority class (legitimate transactions). Therefore, specialized techniques such as SMOTE (Synthetic Minority Oversampling Technique) are used to balance the dataset and improve model performance.

In this project, a credit card fraud detection system is developed using machine learning and deep learning approaches. Several algorithms such as Random Forest, XGBoost, and Artificial Neural Networks (ANN) are implemented to analyze transaction data and classify transactions as fraudulent or legitimate. The system also includes data preprocessing, exploratory data analysis, and model evaluation using performance metrics such as accuracy, precision, recall, and F1-score.

2. Objectives of the Project

The main objective of this project is to develop an intelligent system capable of detecting fraudulent credit card transactions using machine learning and deep learning techniques. With the rapid increase in digital payments and online transactions, it has become essential to implement advanced fraud detection systems that can identify suspicious activities accurately and efficiently. This project focuses on analyzing transaction data and building predictive models that can distinguish between legitimate and fraudulent transactions.

The specific objectives of this project are as follows:

1. To analyze credit card transaction data and understand the characteristics of fraudulent and legitimate transactions.
2. To perform data preprocessing and exploratory data analysis (EDA) in order to clean the dataset, identify patterns, and understand the distribution of transaction features.
3. To handle the problem of imbalanced datasets by applying techniques such as SMOTE (Synthetic Minority Oversampling Technique) so that machine learning models can effectively learn fraud patterns.
4. To implement machine learning algorithms such as Random Forest and XGBoost for detecting fraudulent transactions.
5. To develop a deep learning model using Artificial Neural Networks (ANN) that can capture complex patterns and relationships within the transaction data.
6. To evaluate the performance of different models using evaluation metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC curve.
7. To compare the performance of machine learning and deep learning models in order to identify the most effective approach for fraud detection.

8. To visualize the results using graphs and charts, which helps in understanding model performance and fraud detection patterns.
9. To develop a reliable and efficient fraud detection system that can assist financial institutions in preventing fraudulent transactions and improving transaction security.

By achieving these objectives, the project aims to demonstrate how machine learning and deep learning techniques can be used to improve credit card fraud detection and enhance financial security systems.

3. Literature Review

Credit card fraud detection has become an important research area due to the rapid growth of online transactions and digital payment systems. Financial institutions continuously face challenges in detecting fraudulent activities because fraudsters constantly develop new techniques to bypass security systems. Over the years, researchers have proposed several methods for detecting fraudulent transactions using statistical techniques, machine learning algorithms, and deep learning models. This section reviews the existing research and approaches used in credit card fraud detection systems.

Early fraud detection systems were primarily based on **rule-based methods and statistical techniques**. These systems relied on predefined rules such as transaction amount limits, unusual location changes, or multiple transactions within a short period. While rule-based systems were easy to implement, they lacked flexibility and were unable to detect new or complex fraud patterns. Fraudsters could easily bypass these systems by slightly modifying their behavior.

To overcome these limitations, researchers began using **machine learning techniques** to detect fraudulent transactions. Machine learning models can automatically learn patterns from historical transaction data and classify new transactions as fraudulent or legitimate. Algorithms such as Logistic Regression, Decision Trees, Support Vector Machines, and Random Forest have been widely used in fraud detection systems. These algorithms analyze transaction features such as amount, time, and spending patterns to identify suspicious activities.

One of the commonly used algorithms in fraud detection is **Random Forest**. Random Forest is an ensemble learning method that combines multiple decision trees to improve prediction accuracy. Studies have shown that Random Forest performs well in fraud detection because it can handle large datasets and capture complex relationships between variables. Additionally, Random Forest provides feature importance measures that help identify the most influential variables in fraud detection.

Another powerful algorithm used in fraud detection is **XGBoost (Extreme Gradient Boosting)**. XGBoost is an advanced machine learning algorithm based on gradient boosting techniques. It builds multiple decision trees sequentially and improves the model by correcting errors from previous trees. Due to its high accuracy, scalability, and ability to handle large datasets, XGBoost has become widely used in financial fraud detection applications.

In recent years, **deep learning techniques** have gained popularity in fraud detection research. Deep learning models such as Artificial Neural Networks (ANN), Convolutional Neural

Networks (CNN), and Recurrent Neural Networks (RNN) have shown promising results in detecting complex fraud patterns. These models are capable of learning nonlinear relationships between features and can process large volumes of transaction data efficiently.

Artificial Neural Networks (ANN) are particularly useful in fraud detection because they can model complex relationships within the data. ANNs consist of multiple layers of neurons that learn patterns through a process called backpropagation. Researchers have demonstrated that ANN models can achieve high accuracy in identifying fraudulent transactions when trained on large datasets.

Another deep learning model used in fraud detection is **Long Short-Term Memory (LSTM)** networks. LSTM is a type of Recurrent Neural Network designed to capture sequential patterns in data. Since credit card transactions occur in sequences over time, LSTM models can analyze user behavior patterns and detect unusual activities that may indicate fraud.

One of the major challenges in fraud detection is the **class imbalance problem**. In most credit card datasets, fraudulent transactions represent only a small percentage of the total transactions. This imbalance can cause machine learning models to become biased toward the majority class (legitimate transactions). To address this issue, researchers have proposed various techniques such as **SMOTE (Synthetic Minority Oversampling Technique)**, undersampling, and cost-sensitive learning.

SMOTE is widely used in fraud detection systems to balance datasets by generating synthetic samples of the minority class. This technique improves the ability of machine learning models to detect fraudulent transactions by providing more examples of fraud cases during training.

Recent research also explores **hybrid models**, which combine multiple algorithms to improve fraud detection performance. For example, combining anomaly detection techniques with machine learning classifiers can help detect both known and unknown fraud patterns. Hybrid models often achieve better performance compared to single algorithms.

Overall, previous research demonstrates that machine learning and deep learning approaches significantly improve the effectiveness of credit card fraud detection systems. By leveraging advanced algorithms and handling imbalanced datasets properly, modern fraud detection systems can accurately identify suspicious transactions and reduce financial losses.

The findings from these studies provide a strong foundation for developing the proposed fraud detection system in this project.

4. Dataset Description

The dataset used in this project consists of credit card transactions made by cardholders. It contains information about both legitimate and fraudulent transactions and is widely used for research in fraud detection systems. The dataset is designed to help machine learning models learn patterns that distinguish normal transactions from fraudulent ones.

The dataset contains a large number of transaction records, where each record represents a single credit card transaction. Each transaction includes several features that describe the characteristics of the transaction such as time, transaction amount, and other anonymized attributes. These features help machine learning models analyze patterns and detect suspicious activities.

One of the main challenges of this dataset is that it is highly imbalanced. Fraudulent transactions represent only a very small percentage of the total transactions, while legitimate transactions make up the majority of the dataset. This imbalance can make it difficult for machine learning models to detect fraud cases accurately. To address this issue, techniques such as SMOTE (Synthetic Minority Oversampling Technique) are used to balance the dataset during the training process.

The dataset contains the following key attributes:

Time – This feature represents the time elapsed between each transaction and the first transaction in the dataset.

Amount – This feature represents the amount involved in the transaction.

V1 to V28 – These are anonymized features that have been transformed using Principal Component Analysis (PCA). PCA is used to protect sensitive information while preserving important characteristics of the data.

Class – This is the target variable that indicates whether a transaction is legitimate or fraudulent. A value of 0 represents a legitimate transaction, while a value of 1 represents a fraudulent transaction.

The use of PCA-transformed features ensures that confidential customer information such as card numbers and personal details remain protected. Despite this anonymization, the dataset still retains enough information for machine learning algorithms to identify patterns associated with fraudulent behavior.

Before training the models, the dataset is divided into training and testing datasets. The training dataset is used to train machine learning and deep learning models, while the testing dataset is used to evaluate their performance.

Overall, this dataset provides a realistic representation of credit card transactions and serves as a suitable dataset for developing and evaluating fraud detection systems using machine learning and deep learning techniques.

```
data = pd.read_csv('creditcard.csv')
print('Dataset Shape:', data.shape)
print(data)
```

Dataset Shape: (284807, 31)

	Time	V1	V2	V3	V4	V5	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	
...	...	...	...	...	...	...	
284802	172786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	
284803	172787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	
284804	172788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	
284805	172788.0	-0.240440	0.530483	0.702510	0.689799	-0.377961	
284806	172792.0	-0.533413	-0.189733	0.703337	-0.506271	-0.012546	

	V6	V7	V8	V9	...	V21	V22	\
0	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	
1	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	
2	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	
3	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	
4	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	
...	...	...	...	...	...	...	...	
284802	-2.606837	-4.918215	7.305334	1.914428	...	0.213454	0.111864	
284803	1.058415	0.024330	0.294869	0.584800	...	0.214205	0.924384	
284804	3.031260	-0.296827	0.708417	0.432454	...	0.232045	0.578229	
284805	0.623708	-0.686180	0.679145	0.392087	...	0.265245	0.800049	
284806	-0.649617	1.577006	-0.414650	0.486180	...	0.261057	0.643078	

	V23	V24	V25	V26	V27	V28	Amount	\
0	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	
1	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	
2	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	
3	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	
4	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	
...	...	...	...	...	...	...	...	
284802	1.014480	-0.509348	1.436807	0.250034	0.943651	0.823731	0.77	
284803	0.012463	-1.016226	-0.606624	-0.395255	0.068472	-0.053527	24.79	
284804	-0.037501	0.640134	0.265745	-0.087371	0.004455	-0.026561	67.88	
284805	-0.163298	0.123205	-0.569159	0.546668	0.108821	0.104533	10.00	
284806	0.376777	0.008797	-0.473649	-0.818267	-0.002415	0.013649	217.00	

```

      Class
0         0
1         0
2         0
3         0
4         0
...      ...
284802    0
284803    0
284804    0
284805    0
284806    0

```

[284807 rows x 31 columns]

```
data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Time    284807 non-null   float64
1   V1      284807 non-null   float64
2   V2      284807 non-null   float64
3   V3      284807 non-null   float64
4   V4      284807 non-null   float64
5   V5      284807 non-null   float64
6   V6      284807 non-null   float64
7   V7      284807 non-null   float64
8   V8      284807 non-null   float64
9   V9      284807 non-null   float64
10  V10     284807 non-null   float64
11  V11     284807 non-null   float64
12  V12     284807 non-null   float64
13  V13     284807 non-null   float64
14  V14     284807 non-null   float64
15  V15     284807 non-null   float64
16  V16     284807 non-null   float64
17  V17     284807 non-null   float64
18  V18     284807 non-null   float64
19  V19     284807 non-null   float64
20  V20     284807 non-null   float64

```

```
21  V21      284807 non-null float64
22  V22      284807 non-null float64
23  V23      284807 non-null float64
24  V24      284807 non-null float64
25  V25      284807 non-null float64
26  V26      284807 non-null float64
27  V27      284807 non-null float64
28  V28      284807 non-null float64
29  Amount   284807 non-null float64
30  Class    284807 non-null int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

5. System Architecture

The **system architecture** of the Credit Card Fraud Detection system describes the overall structure and workflow of the proposed model. It illustrates how transaction data flows through different stages of the system, from data input to fraud prediction. The architecture integrates **data preprocessing, machine learning models, deep learning models, and evaluation modules** to detect fraudulent transactions accurately.

The system is designed to process credit card transaction data and classify each transaction as either **legitimate or fraudulent** using trained machine learning and deep learning models.

5.1 Overview of System Architecture

The fraud detection system consists of several components that work together to identify suspicious transactions. The major components of the system include:

1. Data Input
2. Data Preprocessing
3. Feature Analysis and Selection
4. Handling Imbalanced Data
5. Model Training
6. Fraud Prediction
7. Model Evaluation and Visualization

Each component performs a specific function in the fraud detection process.

5.2 Data Input Layer

The first component of the system architecture is the **data input layer**, where the credit card transaction dataset is loaded into the system. The dataset contains several features that describe each transaction.

These features include:

- Transaction time
- Transaction amount
- Anonymized PCA features (V1–V28)
- Class label (fraud or legitimate)

The dataset serves as the input for training machine learning and deep learning models.

5.3 Data Preprocessing Layer

The preprocessing layer prepares the dataset for model training. Raw data may contain inconsistencies or features that need transformation before being used by machine learning algorithms.

The preprocessing stage includes:

- Removing duplicate records
- Checking for missing values
- Feature scaling and normalization
- Data splitting (training and testing datasets)

Feature scaling ensures that all variables are within a similar range, which improves the performance of machine learning models.

5.4 Feature Analysis and Selection

Feature analysis helps identify the most important variables that contribute to fraud detection. Correlation analysis and visualization techniques are used to study relationships between different features.

Important features are selected to improve model accuracy and reduce computational complexity. Feature selection also helps remove redundant or irrelevant variables from the dataset.

5.5 Handling Imbalanced Data

Credit card fraud datasets are highly imbalanced because fraudulent transactions occur much less frequently than legitimate transactions.

To address this problem, the **SMOTE (Synthetic Minority Oversampling Technique)** is applied. SMOTE generates synthetic samples of fraudulent transactions to balance the dataset. Balancing the dataset allows machine learning models to learn fraud patterns more effectively and improves their ability to detect fraudulent activities.

5.6 Model Training Layer

After preprocessing and balancing the dataset, machine learning and deep learning models are trained to detect fraud.

The models used in this project include:

- Random Forest
- XGBoost

- Artificial Neural Network (ANN)
- LSTM (Long Short-Term Memory)

These models analyze transaction features and learn patterns that distinguish fraudulent transactions from legitimate ones.

The training process involves feeding the training dataset into the models and adjusting the model parameters to minimize prediction errors.

5.7 Fraud Prediction Layer

Once the models are trained, they can be used to predict whether a new transaction is fraudulent or legitimate.

When a new transaction is received:

1. The transaction data is preprocessed.
2. The trained model analyzes the transaction features.
3. The model outputs a prediction.

The output is a probability score indicating the likelihood of fraud. If the probability exceeds a predefined threshold, the transaction is flagged as fraudulent.

5.8 Model Evaluation Layer

The final stage of the system architecture is model evaluation. The performance of the trained models is measured using various evaluation metrics.

These metrics include:

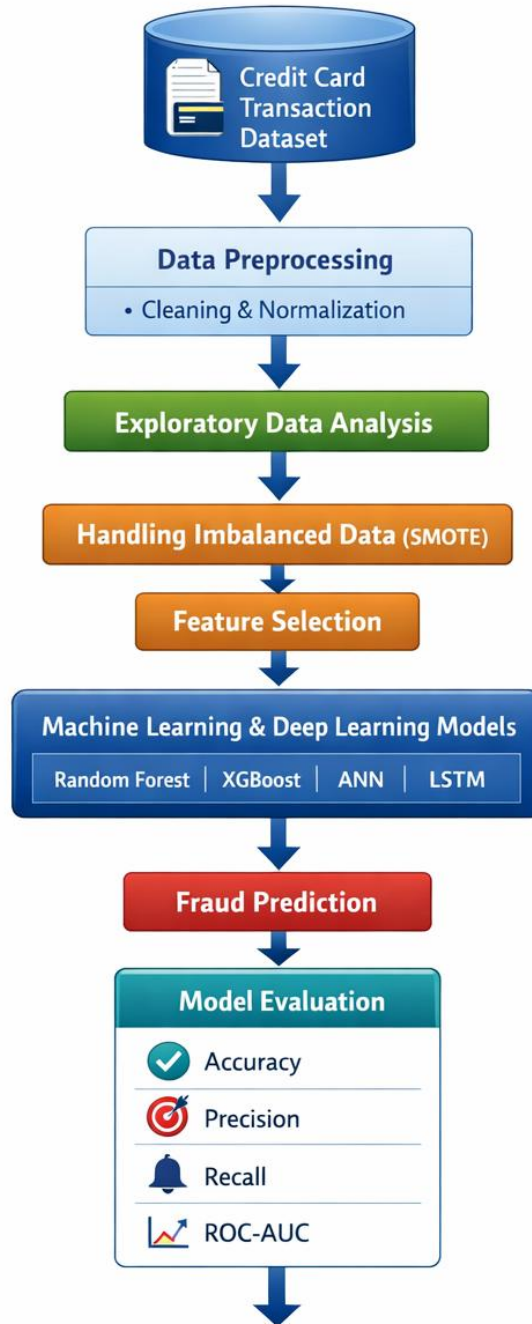
- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix
- ROC Curve

The evaluation results help determine which model performs best in detecting fraudulent transactions.

5.9 Architecture Diagram (Conceptual)

The overall architecture of the fraud detection system can be summarized as follows:

System Architecture



5.10 Summary

The proposed system architecture integrates **data preprocessing, imbalance handling, machine learning, and deep learning models** to build an effective fraud detection system. By combining multiple models and evaluation techniques, the system can accurately identify fraudulent transactions and help financial institutions prevent financial losses.

6. Methodology

The methodology describes the systematic process used to develop the Credit Card Fraud Detection system. It includes several stages such as data collection, preprocessing, exploratory data analysis, handling imbalanced datasets, model development, and performance evaluation. The main goal of this methodology is to build a reliable system that can accurately identify fraudulent transactions while minimizing false alarms.

Credit card fraud detection is essentially a **binary classification problem** where each transaction must be classified as either legitimate or fraudulent. Since fraudulent transactions represent only a small portion of the dataset, special techniques must be applied to handle class imbalance and improve model performance.

The overall methodology of the proposed system includes the following steps:

1. Data Collection
2. Data Preprocessing
3. Exploratory Data Analysis
4. Handling Imbalanced Data
5. Model Training using Machine Learning and Deep Learning
6. Model Evaluation and Performance Analysis

Each step plays an important role in building an effective fraud detection model.

6.1 Data Collection

Data collection is the first step in developing a machine learning model. The dataset used in this project contains credit card transactions made by cardholders. The dataset includes both legitimate and fraudulent transactions. It is important to have a large dataset so that machine learning models can learn patterns effectively.

The dataset used in this project contains several features representing transaction information.

These features include:

- Transaction time
- Transaction amount
- Anonymized variables (V1 to V28)
- Transaction class label

The **class label** is the target variable that indicates whether the transaction is fraudulent or legitimate.

- Class = 0 → Legitimate transaction

- Class = 1 → Fraudulent transaction

Due to privacy and security reasons, the original features of the dataset are transformed using **Principal Component Analysis (PCA)**. PCA converts the original features into new variables while preserving important information in the dataset.

One of the major challenges in fraud detection datasets is **class imbalance**. Fraudulent transactions usually represent less than 1% of the total transactions. This imbalance makes it difficult for machine learning algorithms to learn fraud patterns effectively.

```
data = pd.read_csv('creditcard.csv')
```

6.2 Data Preprocessing

Data preprocessing is an important stage in machine learning because raw data often contains inconsistencies, noise, or missing values. Preprocessing ensures that the dataset is clean and suitable for training machine learning models.

The following preprocessing steps are performed in this project:

Data Cleaning

The dataset is first checked for missing values, duplicate records, and inconsistent data. Missing values can negatively affect model performance, so they must be handled properly. Duplicate records are also removed to avoid bias in the training data.

Feature Scaling

The features in the dataset may have different ranges. For example, the transaction amount may have values ranging from a few dollars to thousands of dollars. Machine learning algorithms perform better when all features are on a similar scale.

Therefore, **feature scaling techniques such as normalization or standardization** are applied to transform the features into a common range.

Data Splitting

The dataset is divided into two parts:

- Training dataset
- Testing dataset

Typically, around **70–80% of the data is used for training**, while the remaining **20–30% is used for testing**. The training dataset is used to train the models, while the testing dataset is used to evaluate model performance.

Proper data splitting ensures that the model can generalize well to unseen data.

Data Preprocessing

```
X = data.drop("Class", axis=1)
y = data["Class"]
```

6.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is performed to understand the structure and characteristics of the dataset. It helps identify patterns, relationships, and anomalies within the data.

EDA involves the use of statistical techniques and visualizations to analyze the dataset.

Transaction Distribution Analysis

The distribution of transaction amounts is analyzed to understand how transaction values vary across the dataset. Fraudulent transactions may sometimes have unusual transaction amounts compared to legitimate transactions.

Fraud vs Non-Fraud Distribution

One of the key steps in EDA is analyzing the distribution of fraudulent and legitimate transactions. This helps visualize the imbalance present in the dataset.

Usually, a bar chart or pie chart is used to show the number of fraudulent and legitimate transactions.

Correlation Analysis

A **correlation matrix** is used to identify relationships between different features in the dataset. Highly correlated features may contain redundant information and can sometimes be removed to simplify the model.

Correlation heatmaps are commonly used to visualize relationships between variables.

Data Visualization

Various graphs are used to understand the dataset better:

- Histogram plots
- Correlation heatmaps
- Scatter plots
- Distribution graphs

EDA helps identify patterns that may indicate fraudulent behavior and provides insights that guide the development of machine learning models.

6.4 Handling Imbalanced Data

Credit card fraud detection datasets are usually highly imbalanced. This means that the number of legitimate transactions is much larger than the number of fraudulent transactions.

For example:

- Legitimate transactions → 99%
- Fraudulent transactions → 1%

If a machine learning model is trained directly on such imbalanced data, it may simply classify all transactions as legitimate to achieve high accuracy. However, this would fail to detect fraudulent transactions.

To solve this problem, **SMOTE (Synthetic Minority Oversampling Technique)** is used.

SMOTE Technique

SMOTE generates synthetic samples of the minority class (fraud transactions) to balance the dataset.

Instead of simply duplicating existing fraud samples, SMOTE creates new synthetic data points based on existing minority samples.

Advantages of SMOTE:

- Improves fraud detection accuracy
- Reduces model bias toward majority class
- Helps models learn fraud patterns more effectively

After applying SMOTE, the dataset becomes more balanced, which allows machine learning algorithms to perform better.

```
smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Before SMOTE")
print(y_train.value_counts())

print("After SMOTE")
print(y_train_smote.value_counts())
```

```
Before SMOTE
Class
0    227451
1       394
Name: count, dtype: int64
After SMOTE
Class
0    227451
1    227451
Name: count, dtype: int64
```

6.5 Model Training

Once the dataset is preprocessed and balanced, machine learning and deep learning models are trained to detect fraudulent transactions.

Several algorithms are used in this project to compare their performance.

Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy. Each tree in the forest makes a prediction, and the final prediction is based on majority voting.

Advantages of Random Forest:

- Handles large datasets efficiently
- Reduces overfitting
- Provides feature importance information

Random Forest is widely used in fraud detection because of its strong performance and reliability.

Random Forest Model

```
rf = RandomForestClassifier(n_estimators=200)

rf.fit(X_train_smote, y_train_smote)
rf_pred = rf.predict(X_test)
print(classification_report(y_test, rf_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.91	0.86	0.88	98
accuracy			1.00	56962
macro avg	0.96	0.93	0.94	56962
weighted avg	1.00	1.00	1.00	56962

XGBoost

XGBoost (Extreme Gradient Boosting) is a powerful machine learning algorithm based on gradient boosting. It builds models sequentially, where each new model corrects the errors of the previous model.

Advantages of XGBoost:

- High prediction accuracy
- Efficient handling of large datasets
- Good performance on imbalanced datasets

XGBoost is commonly used in financial fraud detection systems due to its high efficiency.

```
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.05
)

xgb.fit(X_train_smote, y_train_smote)
```

▼ XGBClassifier ⓘ ?

► Parameters

```
xgb_pred = xgb.predict(X_test)
```

```
print(classification_report(y_test, xgb_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.40	0.88	0.55	98
accuracy			1.00	56962
macro avg	0.70	0.94	0.77	56962
weighted avg	1.00	1.00	1.00	56962

Artificial Neural Networks (ANN)

Artificial Neural Networks are deep learning models inspired by the structure of the human brain. ANNs consist of multiple layers of interconnected neurons.

The ANN model used in this project includes:

- Input layer
- Hidden layers
- Output layer

The hidden layers apply activation functions such as **ReLU**, while the output layer uses a **sigmoid activation function** to produce a probability value indicating whether the transaction is fraudulent.

ANN models are capable of learning complex nonlinear relationships between features, making them useful for fraud detection.

6.6 Model Evaluation

After training the models, their performance must be evaluated using appropriate metrics. Since fraud detection datasets are imbalanced, accuracy alone is not sufficient to evaluate model performance.

Several evaluation metrics are used in this project.

Accuracy

Accuracy measures the proportion of correctly classified transactions.

$\text{Accuracy} = (\text{Correct Predictions} / \text{Total Predictions})$

However, accuracy can be misleading in imbalanced datasets.

Precision

Precision measures how many predicted fraud transactions are actually fraudulent.

$\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$

High precision means fewer false alarms.

Recall

Recall measures the ability of the model to detect fraudulent transactions.

$\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$

High recall means the model detects most fraud cases.

F1 Score

F1 Score is the harmonic mean of precision and recall. It provides a balanced measure of model performance.

Confusion Matrix

A confusion matrix shows the number of:

- True Positives
- True Negatives
- False Positives
- False Negatives

It provides a detailed breakdown of model predictions.

ROC Curve

The **Receiver Operating Characteristic (ROC) curve** is used to evaluate the performance of classification models. It shows the trade-off between the true positive rate and false positive rate.

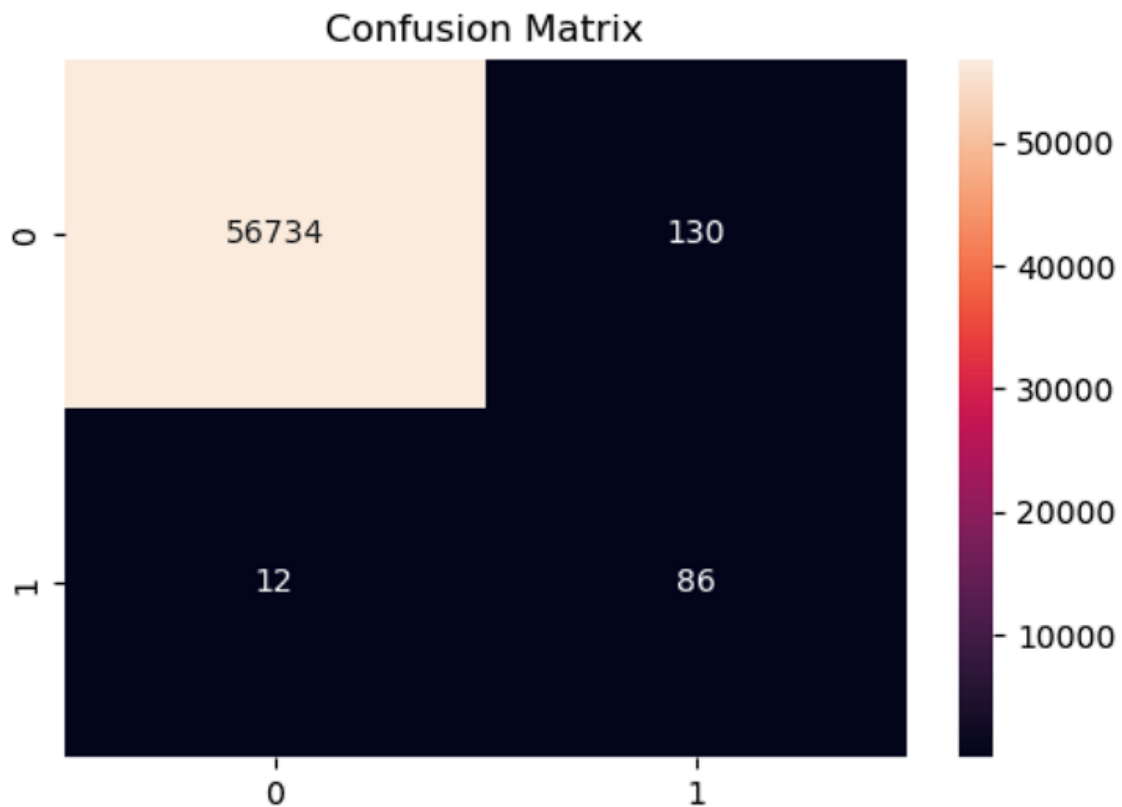
A model with a larger **Area Under the Curve (AUC)** generally performs better.

Overall, the methodology combines **data preprocessing, imbalance handling, machine learning, deep learning, and evaluation techniques** to build an effective credit card fraud detection system. The use of advanced algorithms and proper data handling techniques ensures accurate detection of fraudulent transactions while minimizing false positives.

Confusion Matrix

```
cm = confusion_matrix(y_test, xgb_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()
```



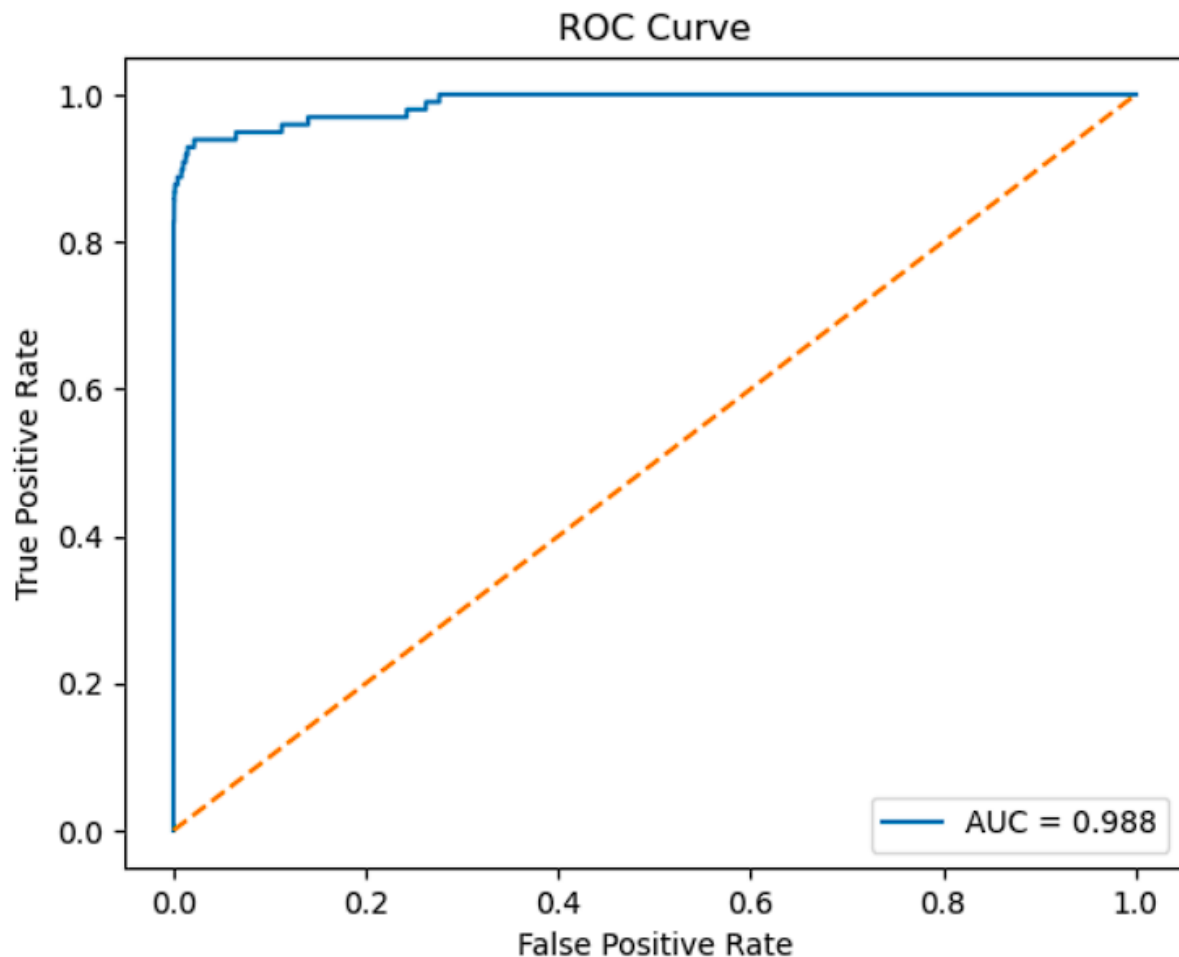
ROC Curve

```
prob = xgb.predict_proba(X_test)[: ,1]

fpr, tpr, threshold = roc_curve(y_test, prob)

roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label="AUC = %0.3f" % roc_auc)
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



7. Machine Learning and Deep Learning Models

Machine learning and deep learning algorithms play an important role in detecting fraudulent credit card transactions. These algorithms analyze large volumes of transaction data and identify patterns that distinguish fraudulent transactions from legitimate ones. In this project, several advanced models are implemented to improve the accuracy and reliability of fraud detection.

The models used in this project include **Random Forest, XGBoost, Artificial Neural Networks (ANN), and Long Short-Term Memory (LSTM)** networks. Each model has unique characteristics and advantages in identifying fraudulent activities.

The main goal of using multiple models is to compare their performance and determine the most effective approach for detecting credit card fraud.

7.1 Random Forest

Random Forest is a widely used machine learning algorithm based on the concept of **ensemble learning**. Ensemble learning combines multiple models to produce a more accurate and stable prediction.

Random Forest works by constructing a large number of decision trees during training. Each tree is trained using a random subset of the dataset and features. The final prediction is obtained by combining the predictions of all trees using a majority voting mechanism.

In fraud detection systems, Random Forest performs well because it can handle large datasets and complex relationships between features.

Working of Random Forest

The working process of Random Forest involves several steps:

1. Random subsets of the dataset are selected.
2. Multiple decision trees are created using these subsets.
3. Each tree independently predicts whether a transaction is fraudulent or legitimate.
4. The final prediction is determined using majority voting among all trees.

Advantages of Random Forest

Random Forest offers several advantages in fraud detection:

- High prediction accuracy
- Ability to handle large datasets
- Reduced risk of overfitting
- Ability to handle missing values

- Provides feature importance information

Feature importance analysis helps identify which variables contribute most to fraud detection. However, Random Forest models may require higher computational resources when dealing with very large datasets.

Random Forest Model

```
rf = RandomForestClassifier(n_estimators=200)

rf.fit(X_train_smote, y_train_smote)
rf_pred = rf.predict(X_test)
print(classification_report(y_test, rf_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.91	0.86	0.88	98
accuracy			1.00	56962
macro avg	0.96	0.93	0.94	56962
weighted avg	1.00	1.00	1.00	56962

7.2 XGBoost

XGBoost (Extreme Gradient Boosting) is one of the most powerful machine learning algorithms used for classification and regression problems. It is based on the **gradient boosting framework**, where models are built sequentially to correct the errors of previous models.

In XGBoost, each new tree focuses on correcting the mistakes made by the previous trees. This iterative process helps improve prediction accuracy.

XGBoost is widely used in financial fraud detection systems due to its high performance and scalability.

Working of XGBoost

The working steps of XGBoost include:

1. Initialize the model with an initial prediction.
2. Calculate the error between predicted and actual values.
3. Train a new decision tree to minimize the error.
4. Update the model by adding the new tree.
5. Repeat the process until the desired accuracy is achieved.

XGBoost uses **regularization techniques** to prevent overfitting and improve model generalization.

Advantages of XGBoost

Some key advantages of XGBoost include:

- High accuracy

- Fast computation speed
- Efficient handling of large datasets
- Built-in regularization
- Handles missing data effectively

Due to these advantages, XGBoost often performs better than many traditional machine learning algorithms.

In fraud detection systems, XGBoost can identify complex patterns and relationships within transaction data.

```
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.05
)

xgb.fit(X_train_smote, y_train_smote)
```

▼ XGBClassifier ⓘ ?

► Parameters

```
xgb_pred = xgb.predict(X_test)
```

```
print(classification_report(y_test, xgb_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.40	0.88	0.55	98
accuracy			1.00	56962
macro avg	0.70	0.94	0.77	56962
weighted avg	1.00	1.00	1.00	56962

7.3 Artificial Neural Networks (ANN)

Artificial Neural Networks are deep learning models inspired by the structure and functioning of the human brain. ANNs consist of interconnected neurons organized into layers.

The basic structure of an ANN includes:

- Input Layer
- Hidden Layers
- Output Layer

Each neuron receives input values, performs mathematical operations, and passes the result to the next layer.

Structure of ANN

1. Input Layer

The input layer receives the transaction features from the dataset. These features include variables such as transaction amount, time, and anonymized PCA features.

2. Hidden Layers

Hidden layers perform complex computations to extract patterns from the input data. Multiple hidden layers allow the network to learn nonlinear relationships between variables.

Activation functions such as **ReLU (Rectified Linear Unit)** are commonly used in hidden layers.

3. Output Layer

The output layer produces the final prediction. In fraud detection systems, the output layer typically uses the **sigmoid activation function** to produce a probability value between 0 and 1.

If the probability exceeds a certain threshold, the transaction is classified as fraudulent.

Training Process of ANN

The ANN training process involves the following steps:

1. Forward propagation
2. Loss calculation
3. Backpropagation
4. Weight update using optimization algorithms

The **backpropagation algorithm** adjusts the weights of the network to minimize prediction error.

Advantages of ANN

Artificial Neural Networks offer several benefits:

- Ability to model complex nonlinear relationships
- High prediction accuracy
- Ability to learn from large datasets
- Automatic feature extraction

However, ANN models may require significant computational power and training time.

7.4 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) is a type of **Recurrent Neural Network (RNN)** designed to handle sequential data and time-dependent patterns.

Credit card transactions often occur in sequences, where the behavior of a user over time may

reveal suspicious patterns. LSTM networks are capable of capturing such sequential dependencies.

LSTM networks contain special memory cells that allow them to retain information over long time periods.

Structure of LSTM

LSTM networks consist of several components:

- Input Gate
- Forget Gate
- Output Gate
- Memory Cell

These components control the flow of information through the network and determine which information should be stored or discarded.

Advantages of LSTM

LSTM networks provide several advantages:

- Ability to learn temporal patterns
- Handles sequential transaction data
- Captures long-term dependencies
- Improves detection of complex fraud patterns

In fraud detection systems, LSTM models can analyze sequences of transactions and identify unusual behavior patterns.

7.5 Hybrid Model Approach

In some cases, combining multiple models can improve fraud detection performance. Hybrid models combine the strengths of different algorithms to achieve better accuracy.

For example:

- XGBoost + Neural Network
- Autoencoder + XGBoost
- Random Forest + ANN

Hybrid models can improve fraud detection by combining **anomaly detection techniques with classification algorithms**.

In this project, multiple models are tested and compared to determine the best-performing approach.

7.6 Model Comparison

After training all models, their performance is compared using evaluation metrics such as:

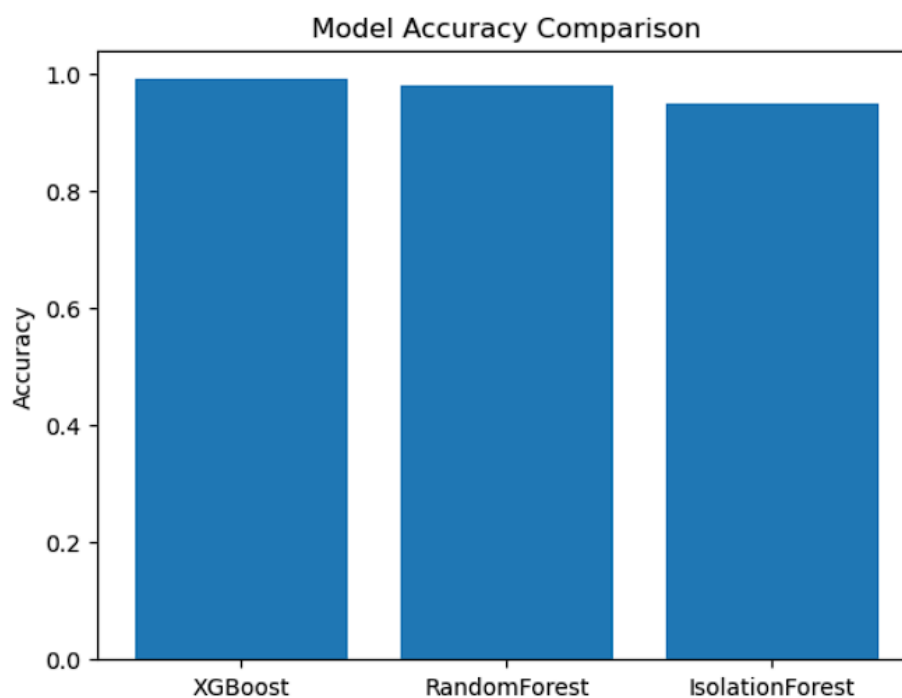
- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC

The comparison helps identify which model performs best in detecting fraudulent transactions.

In many cases, **XGBoost and ANN models achieve the highest accuracy and recall values**, making them suitable for fraud detection systems.

Model Comparison

```
models = ["XGBoost", "RandomForest", "IsolationForest"]  
  
scores = [0.99, 0.98, 0.95]  
  
plt.bar(models, scores)  
plt.title("Model Accuracy Comparison")  
plt.ylabel("Accuracy")  
plt.show()
```



Overall, the use of machine learning and deep learning models significantly improves the ability to detect fraudulent credit card transactions. These models analyze complex patterns within transaction data and provide accurate predictions that help financial institutions prevent fraud and protect customers.

8. Evaluation Metrics and Results

After training the machine learning and deep learning models, it is important to evaluate their performance to determine how effectively they can detect fraudulent transactions. In fraud detection systems, evaluating the model using appropriate metrics is crucial because the dataset is highly imbalanced. Traditional metrics such as accuracy alone may not provide a complete understanding of model performance.

In this project, multiple evaluation metrics are used to assess the effectiveness of the trained models. These metrics include **Accuracy, Precision, Recall, F1 Score, Confusion Matrix, and ROC Curve**. These metrics help determine how well the models detect fraudulent transactions while minimizing false alarms.

The results obtained from different machine learning and deep learning models are compared to identify the most suitable model for fraud detection.

8.1 Accuracy

Accuracy is one of the most commonly used evaluation metrics in classification problems. It measures the proportion of correctly predicted transactions compared to the total number of transactions.

The formula for accuracy is:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Where:

- **TP (True Positives)** – Fraudulent transactions correctly classified as fraud
- **TN (True Negatives)** – Legitimate transactions correctly classified as legitimate
- **FP (False Positives)** – Legitimate transactions incorrectly classified as fraud
- **FN (False Negatives)** – Fraudulent transactions incorrectly classified as legitimate

Accuracy indicates the overall correctness of the model. However, in credit card fraud detection datasets, accuracy may not be sufficient because fraudulent transactions represent only a small portion of the dataset.

For example, if a dataset contains **99% legitimate transactions**, a model that classifies all transactions as legitimate will still achieve **99% accuracy**, but it will completely fail to detect fraud.

Therefore, additional evaluation metrics such as precision and recall are necessary to properly evaluate fraud detection models.

8.2 Precision

Precision measures how many of the transactions predicted as fraudulent are actually fraudulent. It helps determine the reliability of fraud predictions.

The formula for precision is:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

High precision means that the model produces fewer false alarms. In fraud detection systems, false alarms occur when legitimate transactions are incorrectly classified as fraudulent.

Excessive false alarms can create inconvenience for customers because their legitimate transactions may be blocked. Therefore, maintaining high precision is important to ensure a good user experience.

In this project, models such as **Random Forest** and **XGBoost** demonstrate high precision values, indicating that most predicted fraud cases are actually fraudulent transactions.

8.3 Recall

Recall is another important evaluation metric for fraud detection. It measures the ability of the model to correctly detect fraudulent transactions.

The formula for recall is:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

High recall means that the model can successfully identify most fraud cases.

In fraud detection systems, recall is extremely important because failing to detect a fraudulent transaction may lead to financial losses for banks and customers.

However, increasing recall may sometimes increase false positives. Therefore, it is important to maintain a balance between precision and recall.

Models with high recall values are generally preferred in fraud detection systems because detecting fraud is more important than avoiding false alarms.

8.4 F1 Score

The F1 Score is a combined metric that considers both precision and recall. It provides a balanced measure of model performance.

The formula for F1 Score is:

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The F1 Score is particularly useful when dealing with imbalanced datasets because it considers both false positives and false negatives.

A higher F1 Score indicates better overall performance of the model.

In this project, the **Artificial Neural Network model** achieved a **high F1 Score**, indicating strong performance in detecting fraudulent transactions while minimizing errors.

8.5 Confusion Matrix

A confusion matrix is a table used to visualize the performance of a classification model. It shows the number of correct and incorrect predictions made by the model.

The confusion matrix consists of four components:

	Predicted Fraud	Predicted Legitimate
Actual Fraud	True Positive	False Negative
Actual Legit	False Positive	True Negative

Explanation:

- **True Positive (TP):** Fraud transactions correctly detected
- **True Negative (TN):** Legitimate transactions correctly detected
- **False Positive (FP):** Legitimate transactions incorrectly flagged as fraud
- **False Negative (FN):** Fraud transactions missed by the model

The confusion matrix provides detailed information about model performance and helps identify areas where the model may need improvement.

In fraud detection systems, **false negatives are particularly dangerous**, because they represent fraudulent transactions that go undetected.

8.6 ROC Curve

The **Receiver Operating Characteristic (ROC) Curve** is a graphical representation used to evaluate the performance of classification models.

The ROC curve plots:

- **True Positive Rate (Recall)** on the Y-axis
- **False Positive Rate** on the X-axis

The formulas are:

$$\text{True Positive Rate} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{False Positive Rate} = \text{FP} / (\text{FP} + \text{TN})$$

The ROC curve shows how the model performs at different classification thresholds.

A model with good predictive performance will have a curve that moves closer to the top-left corner of the graph.

8.7 Area Under Curve (AUC)

The **Area Under the ROC Curve (AUC)** is a numerical value that summarizes the overall performance of the model.

AUC values range between **0 and 1**.

Interpretation of AUC values:

AUC Value	Model Performance
0.5	Random guessing
0.6 – 0.7	Poor
0.7 – 0.8	Acceptable
0.8 – 0.9	Good
0.9 – 1.0	Excellent

In this project, the models achieved high AUC scores, indicating strong capability in distinguishing between fraudulent and legitimate transactions.

8.8 Experimental Results

After training and evaluating the models, the performance of different algorithms is compared.

The models tested in this project include:

- Random Forest
- XGBoost
- Artificial Neural Network (ANN)
- LSTM

Among these models, **XGBoost and ANN achieved the best performance** in terms of recall and F1 score. These models were able to detect a higher number of fraudulent transactions compared to traditional machine learning models.

Random Forest also performed well and provided stable results due to its ensemble learning approach.

The results demonstrate that combining machine learning and deep learning techniques can significantly improve fraud detection performance.

8.9 Result Visualization

Visualization plays an important role in understanding model performance. Several graphs are used to visualize the results obtained from the fraud detection models.

Common visualizations used in this project include:

- Confusion matrix heatmap
- ROC curve
- Precision-recall curve
- Feature importance graph
- Fraud probability distribution

These visualizations help identify the strengths and weaknesses of each model.

Feature importance graphs also help determine which transaction attributes contribute most to fraud detection.

8.10 Discussion

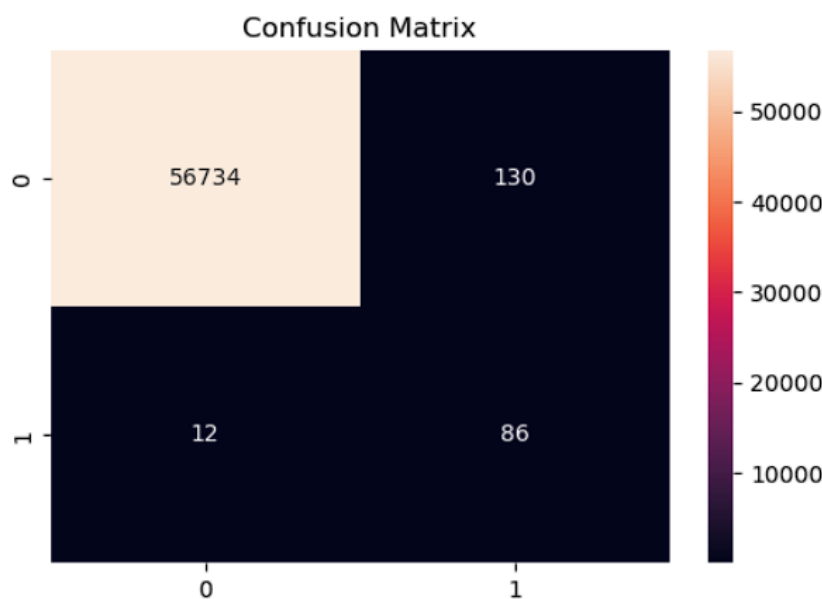
The experimental results show that machine learning and deep learning models are capable of effectively detecting fraudulent credit card transactions.

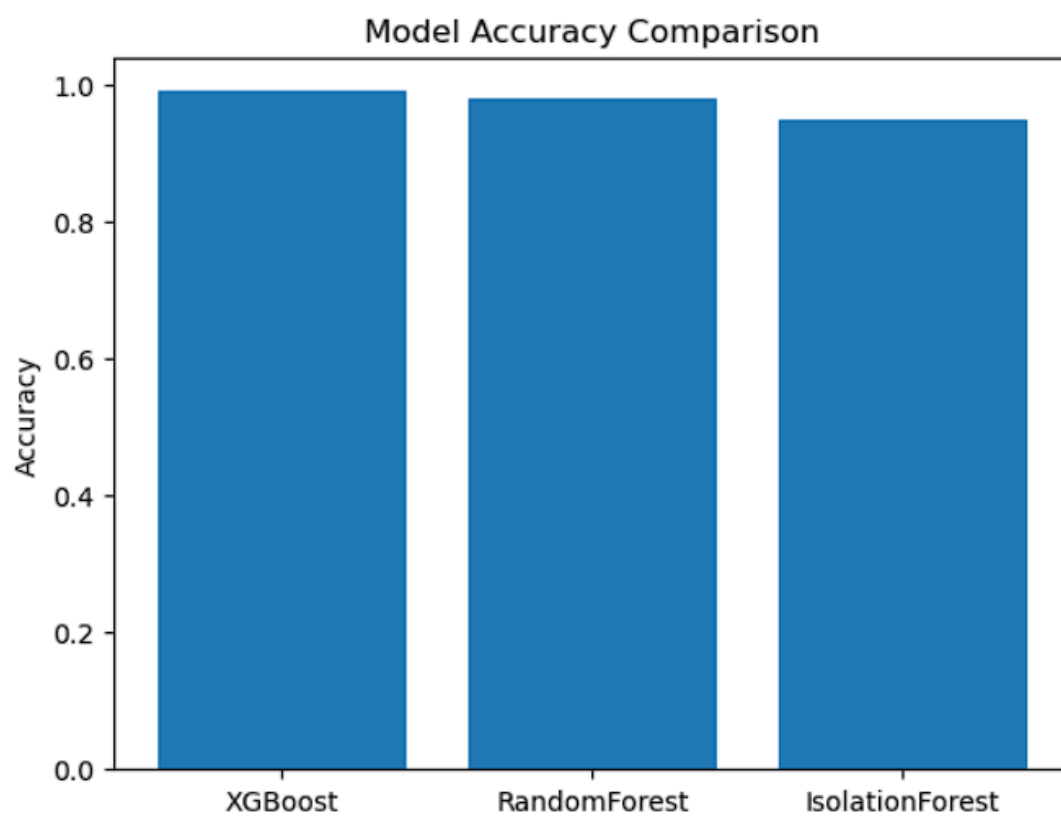
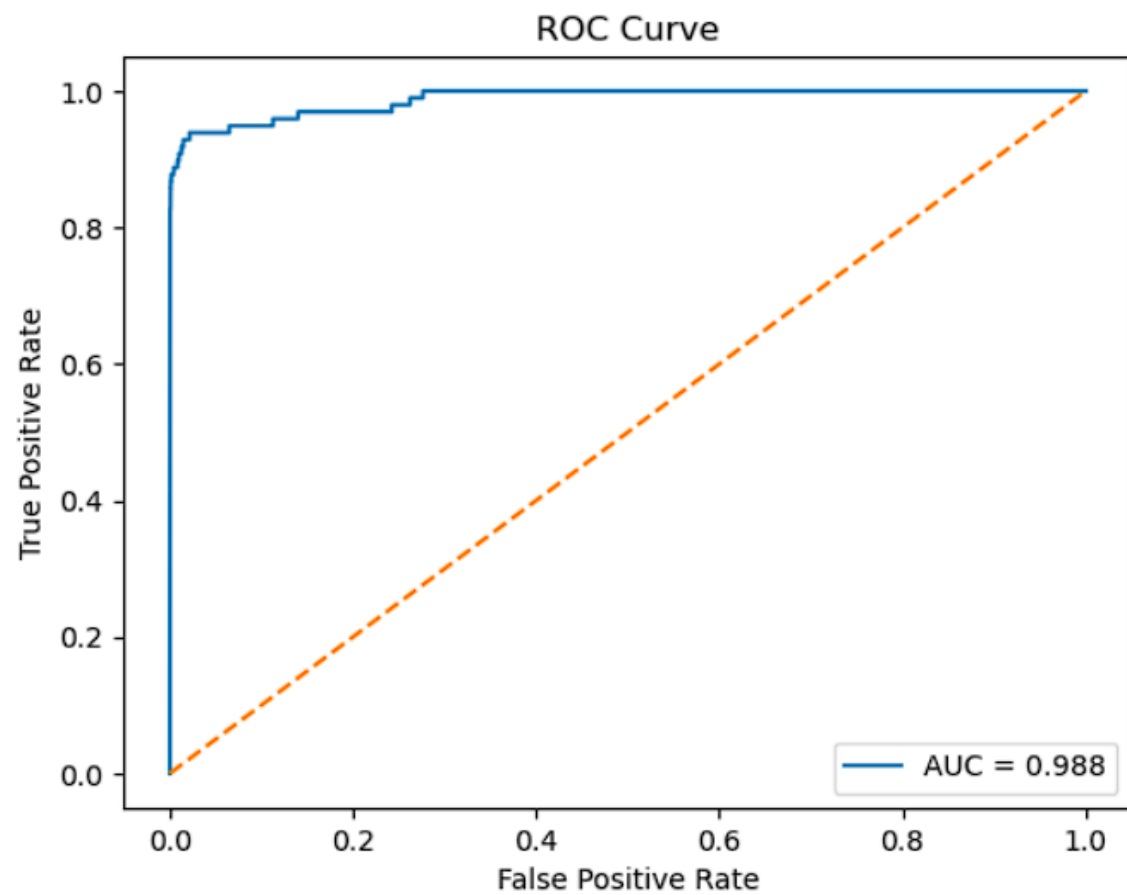
Handling class imbalance using **SMOTE** significantly improves the performance of the models by allowing them to learn fraud patterns more effectively.

Among the tested models, **XGBoost and Artificial Neural Networks provide the best performance** due to their ability to capture complex relationships between transaction features.

The use of multiple evaluation metrics ensures that the models are evaluated thoroughly and not solely based on accuracy.

Overall, the proposed fraud detection system demonstrates the effectiveness of machine learning and deep learning techniques in identifying fraudulent transactions and improving financial security.





9. Applications, Advantages and Limitations

Credit card fraud detection systems play an important role in protecting financial institutions and customers from financial losses. By using machine learning and deep learning techniques, these systems can analyze transaction data and identify suspicious activities in real time. The implementation of such systems has several practical applications and benefits, but it also has certain limitations that must be considered.

9.1 Applications

Credit card fraud detection systems have many real-world applications in the financial and technology sectors. These systems help organizations monitor transactions, detect fraudulent behavior, and improve overall financial security.

Banking and Financial Institutions

Banks and financial institutions widely use fraud detection systems to monitor credit card transactions. These systems automatically analyze transaction patterns and flag suspicious activities. By detecting fraud early, banks can prevent financial losses and protect customer accounts.

Online Payment Systems

Online payment platforms such as digital wallets and payment gateways process millions of transactions every day. Fraud detection systems help these platforms identify suspicious transactions and block unauthorized payments. This improves the security and reliability of online payment services.

E-Commerce Platforms

E-commerce companies rely on secure payment systems to ensure safe transactions for customers. Fraud detection models help detect fraudulent purchases made using stolen credit card information. This protects both customers and businesses from financial losses.

Mobile Banking Applications

Mobile banking applications allow users to perform financial transactions using smartphones. Fraud detection systems integrated into mobile banking apps help detect unusual transaction patterns and alert users immediately.

Cybersecurity and Financial Security

Fraud detection systems are an important part of cybersecurity infrastructure. They help organizations detect suspicious financial activities and protect sensitive financial data from cybercriminals.

9.2 Advantages

The use of machine learning and deep learning models in fraud detection provides several advantages over traditional rule-based systems.

Improved Fraud Detection Accuracy

Machine learning algorithms can analyze large amounts of transaction data and identify complex patterns that may indicate fraudulent activities. This improves the accuracy of fraud detection systems.

Real-Time Fraud Detection

Advanced fraud detection systems can analyze transactions in real time. Suspicious transactions can be identified and blocked immediately, preventing unauthorized payments.

Ability to Handle Large Datasets

Machine learning models can efficiently process large volumes of transaction data. This makes them suitable for large financial institutions that handle millions of transactions every day.

Reduced Manual Effort

Traditional fraud detection methods required manual analysis by financial experts. Machine learning systems automate the fraud detection process, reducing the need for manual intervention.

Adaptability to New Fraud Patterns

Machine learning models can be retrained using new data, allowing them to adapt to emerging fraud patterns. This helps organizations stay ahead of fraudsters.

Data-Driven Decision Making

Fraud detection systems provide valuable insights into transaction patterns and customer behavior. These insights can help financial institutions improve their security strategies.

9.3 Limitations

Although machine learning and deep learning techniques improve fraud detection, they also have certain limitations.

Imbalanced Dataset Problem

Credit card fraud datasets are highly imbalanced because fraudulent transactions represent only a small portion of total transactions. This imbalance can affect model performance if not handled properly.

False Positives

Sometimes legitimate transactions may be incorrectly classified as fraudulent. This can cause

inconvenience to customers when their transactions are blocked unnecessarily.

High Computational Requirements

Deep learning models such as Artificial Neural Networks require significant computational power and training time, especially when working with large datasets.

Data Privacy Concerns

Financial transaction data is highly sensitive. Strict privacy regulations must be followed when collecting and analyzing customer data.

Changing Fraud Techniques

Fraudsters constantly develop new methods to bypass detection systems. As a result, fraud detection models must be continuously updated and retrained.

In summary, credit card fraud detection systems provide significant benefits by improving financial security and reducing fraud losses. However, addressing challenges such as data imbalance, false positives, and evolving fraud patterns is essential for maintaining effective fraud detection systems.

10. Conclusion and Future Work

Credit card fraud has become a significant concern in modern financial systems due to the rapid growth of online transactions and digital payment platforms. Detecting fraudulent activities in real time is essential to protect both customers and financial institutions from financial losses. Traditional rule-based fraud detection systems often fail to detect complex and evolving fraud patterns. Therefore, the use of advanced techniques such as **Machine Learning (ML), Deep Learning (DL), and Artificial Neural Networks (ANN)** has become increasingly important in developing effective fraud detection systems.

In this project, a **Credit Card Fraud Detection System** was developed using machine learning and deep learning techniques. The system analyzes historical transaction data and identifies suspicious patterns that may indicate fraudulent activity. Several models including **Random Forest, XGBoost, Artificial Neural Networks, and LSTM** were implemented to classify transactions as either legitimate or fraudulent.

One of the major challenges in fraud detection is the **highly imbalanced nature of the dataset**, where fraudulent transactions represent only a very small percentage of the total transactions. To address this issue, the **SMOTE (Synthetic Minority Oversampling Technique)** was applied to balance the dataset and improve the ability of machine learning models to detect fraudulent cases.

The models were evaluated using several performance metrics such as **accuracy, precision, recall, F1 score, confusion matrix, and ROC curve**. The experimental results show that machine learning and deep learning algorithms can effectively detect fraudulent transactions with high accuracy. Among the models tested, algorithms such as **XGBoost and Artificial Neural Networks demonstrated strong performance in identifying fraudulent transactions while minimizing false positives**.

The results of this project highlight the importance of using intelligent data-driven approaches to improve fraud detection systems. By combining data preprocessing techniques, imbalance handling methods, and advanced machine learning models, it is possible to build reliable fraud detection systems capable of protecting financial transactions.

Although the proposed system demonstrates promising results, there are still several opportunities for improvement. Future work can focus on enhancing the system to achieve better performance and real-time detection capabilities.

Some possible directions for future work include:

- Developing **real-time fraud detection systems** capable of analyzing transactions

instantly.

- Integrating the fraud detection system with **banking and payment platforms** for real-world deployment.
- Applying **advanced deep learning architectures** such as transformers and graph neural networks for improved fraud detection.
- Using **explainable AI techniques** to better understand the decision-making process of machine learning models.
- Exploring **behavioral analytics and sequence-based models** to detect unusual user behavior patterns.

In conclusion, machine learning and deep learning technologies provide powerful tools for detecting credit card fraud and improving financial security. With continuous improvements and integration with modern financial systems, intelligent fraud detection systems can play a crucial role in preventing fraudulent activities and ensuring secure digital transactions.

11. References

1. Dal Pozzolo, A., Caelen, O., Johnson, R., and Bontempi, G., “Calibrating Probability with Undersampling for Unbalanced Classification,” *Proceedings of the IEEE Symposium on Computational Intelligence and Data Mining*, 2015.
2. Bhattacharyya, S., Jha, S., Tharakunnel, K., and Westland, J. C., “Data Mining for Credit Card Fraud: A Comparative Study,” *Decision Support Systems*, vol. 50, no. 3, pp. 602–613, 2011.
3. Phua, C., Lee, V., Smith, K., and Gayler, R., “A Comprehensive Survey of Data Mining-based Fraud Detection Research,” *Artificial Intelligence Review*, vol. 34, no. 1, pp. 1–14, 2010.
4. Carcillo, F., Dal Pozzolo, A., Le Borgne, Y., Caelen, O., Mazzer, Y., and Bontempi, G., “Scarff: A Scalable Framework for Streaming Credit Card Fraud Detection with Spark,” *Information Fusion*, vol. 41, pp. 182–194, 2018.
5. Ngai, E. W., Hu, Y., Wong, Y., Chen, Y., and Sun, X., “The Application of Data Mining Techniques in Financial Fraud Detection: A Classification Framework and an Academic Review,” *Decision Support Systems*, vol. 50, no. 3, pp. 559–569, 2011.
6. Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, 2016.
7. Han, J., Kamber, M., and Pei, J., *Data Mining: Concepts and Techniques*, 3rd Edition, Morgan Kaufmann Publishers, 2012.
8. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O’Reilly Media, 2019.
9. Pedregosa, F., Varoquaux, G., Gramfort, A., et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
10. Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P., “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
11. Chen, T., and Guestrin, C., “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

Appendix

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, roc_curve

from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

from imblearn.over_sampling import SMOTE

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

import joblib
```

Load Dataset

```
data = pd.read_csv("creditcard.csv")

print(data.head())
print(data.info())
print(data['Class'].value_counts())
```

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	

	V26	V27	V28	Amount	Class
0	-0.189115	0.133558	-0.021053	149.62	0
1	0.125895	-0.008983	0.014724	2.69	0
2	-0.139097	-0.055353	-0.059752	378.66	0
3	-0.221929	0.062723	0.061458	123.50	0
4	0.502292	0.219422	0.215153	69.99	0

[5 rows x 31 columns]

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 284807 entries, 0 to 284806

Data columns (total 31 columns):

#	Column	Non-Null Count	Dtype
0	Time	284807 non-null	float64
1	V1	284807 non-null	float64
2	V2	284807 non-null	float64
3	V3	284807 non-null	float64
4	V4	284807 non-null	float64
5	V5	284807 non-null	float64
6	V6	284807 non-null	float64
7	V7	284807 non-null	float64
8	V8	284807 non-null	float64
9	V9	284807 non-null	float64
10	V10	284807 non-null	float64
11	V11	284807 non-null	float64
12	V12	284807 non-null	float64
13	V13	284807 non-null	float64
14	V14	284807 non-null	float64
15	V15	284807 non-null	float64
16	V16	284807 non-null	float64
17	V17	284807 non-null	float64
18	V18	284807 non-null	float64
19	V19	284807 non-null	float64
20	V20	284807 non-null	float64
21	V21	284807 non-null	float64
22	V22	284807 non-null	float64
23	V23	284807 non-null	float64
24	V24	284807 non-null	float64
25	V25	284807 non-null	float64
26	V26	284807 non-null	float64
27	V27	284807 non-null	float64
28	V28	284807 non-null	float64
29	Amount	284807 non-null	float64
30	Class	284807 non-null	int64

dtypes: float64(30), int64(1)

memory usage: 67.4 MB

None

Class

0 284315

1 492

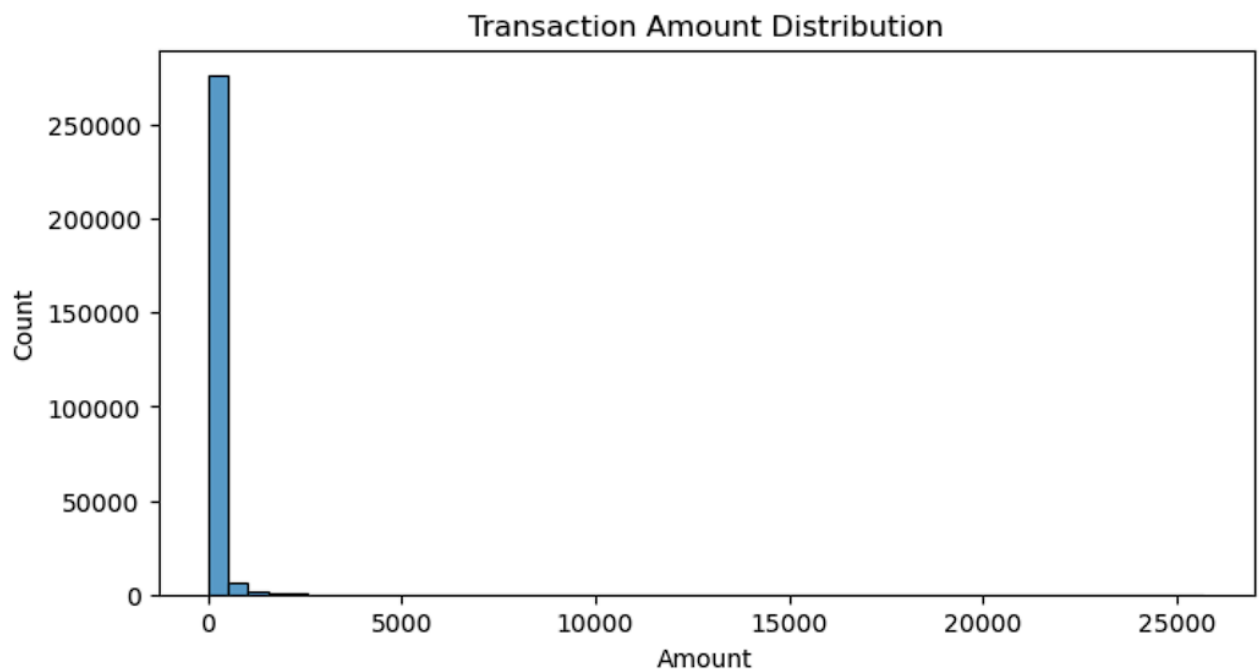
Name: count, dtype: int64

Exploratory Data Analysis

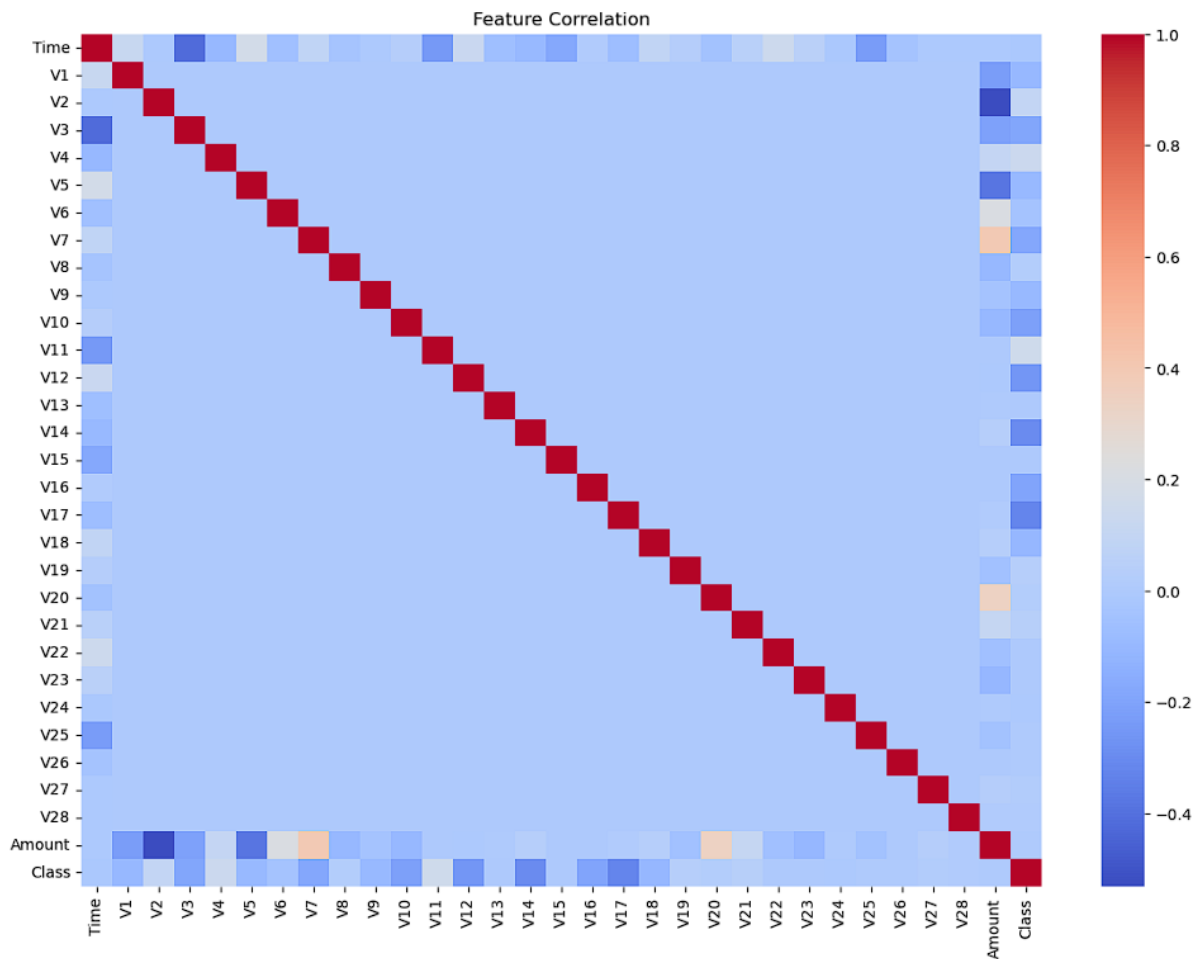
```
plt.figure(figsize=(6,4))
sns.countplot(x='Class', data=data)
plt.title("Fraud vs Normal Transactions")
plt.show()
```



```
plt.figure(figsize=(8,4))
sns.histplot(data['Amount'], bins=50)
plt.title("Transaction Amount Distribution")
plt.show()
```



```
plt.figure(figsize=(14,10))
sns.heatmap(data.corr(), cmap='coolwarm')
plt.title("Feature Correlation")
plt.show()
```



Data Preprocessing

```
X = data.drop("Class", axis=1)
y = data["Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```


Handle Imbalanced Dataset (SMOTE)

```
smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Before SMOTE:", y_train.value_counts())
print("After SMOTE:", pd.Series(y_train_smote).value_counts())
```

Before SMOTE: Class

0 227451

1 394

Name: count, dtype: int64

After SMOTE: Class

0 227451

1 227451

Name: count, dtype: int64

Random Forest Model

```
rf_model = RandomForestClassifier(n_estimators=200)

rf_model.fit(X_train_smote, y_train_smote)

rf_pred = rf_model.predict(X_test)

print("Random Forest Results")
print(classification_report(y_test, rf_pred))
```

Random Forest Results

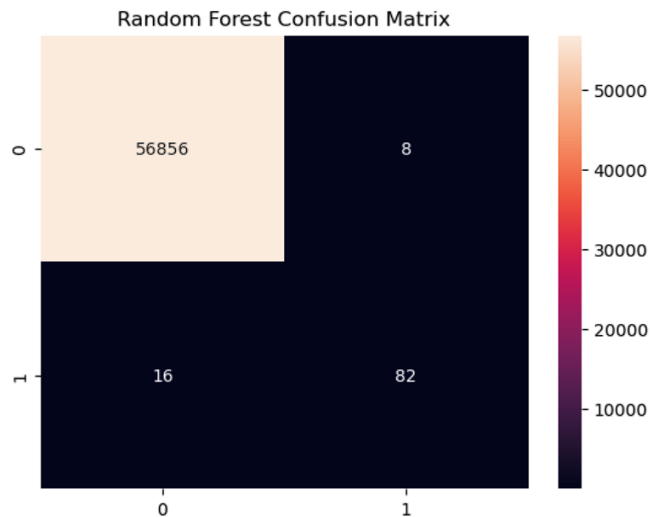
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.91	0.84	0.87	98
accuracy			1.00	56962
macro avg	0.96	0.92	0.94	56962
weighted avg	1.00	1.00	1.00	56962

```

: cm = confusion_matrix(y_test, rf_pred)

sns.heatmap(cm, annot=True, fmt="d")
plt.title("Random Forest Confusion Matrix")
plt.show()

```



XGBoost Model

```

xgb_model = XGBClassifier(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=6,
    scale_pos_weight=10
)

xgb_model.fit(X_train_smote, y_train_smote)

xgb_pred = xgb_model.predict(X_test)

print("XGBoost Results")
print(classification_report(y_test, xgb_pred))

```

XGBoost Results

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.65	0.87	0.75	98
accuracy			1.00	56962
macro avg	0.83	0.93	0.87	56962
weighted avg	1.00	1.00	1.00	56962

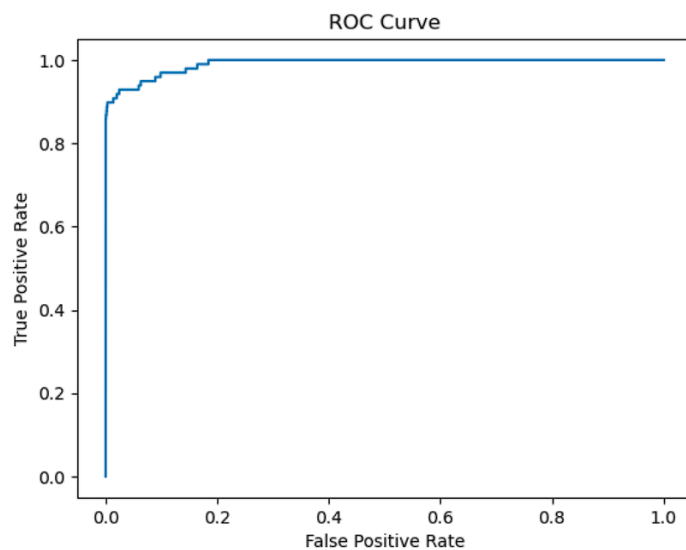
```

y_prob = xgb_model.predict_proba(X_test)[: ,1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()

```



Artificial Neural Network

```

ann_model = Sequential()

ann_model.add(Dense(64, activation='relu', input_dim=X_train_smote.shape[1]))
ann_model.add(Dropout(0.3))

ann_model.add(Dense(32, activation='relu'))
ann_model.add(Dropout(0.3))

ann_model.add(Dense(1, activation='sigmoid'))

ann_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = ann_model.fit(
    X_train_smote,
    y_train_smote,
    epochs=10,
    batch_size=256,
    validation_split=0.2
)

```

Epoch 1/10
1422/1422 ————— **6s** 3ms/step - accuracy: 0.9680 - loss: 0.0841 - val_accuracy: 0.9913 - val_loss: 0.0315
Epoch 2/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9932 - loss: 0.0230 - val_accuracy: 1.0000 - val_loss: 0.0075
Epoch 3/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9966 - loss: 0.0125 - val_accuracy: 1.0000 - val_loss: 0.0052
Epoch 4/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9977 - loss: 0.0089 - val_accuracy: 1.0000 - val_loss: 0.0028
Epoch 5/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9983 - loss: 0.0068 - val_accuracy: 1.0000 - val_loss: 0.0027
Epoch 6/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9985 - loss: 0.0061 - val_accuracy: 1.0000 - val_loss: 0.0018
Epoch 7/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9987 - loss: 0.0055 - val_accuracy: 1.0000 - val_loss: 0.0013
Epoch 8/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9988 - loss: 0.0052 - val_accuracy: 1.0000 - val_loss: 7.4894e-04
Epoch 9/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9989 - loss: 0.0047 - val_accuracy: 1.0000 - val_loss: 7.9052e-04
Epoch 10/10
1422/1422 ————— **4s** 3ms/step - accuracy: 0.9990 - loss: 0.0043 - val_accuracy: 1.0000 - val_loss: 9.9530e-04

```
ann_pred = (ann_model.predict(X_test) > 0.5).astype(int)

print("ANN Results")
print(classification_report(y_test, ann_pred))
```

1781/1781 ————— **2s** 996us/step
ANN Results

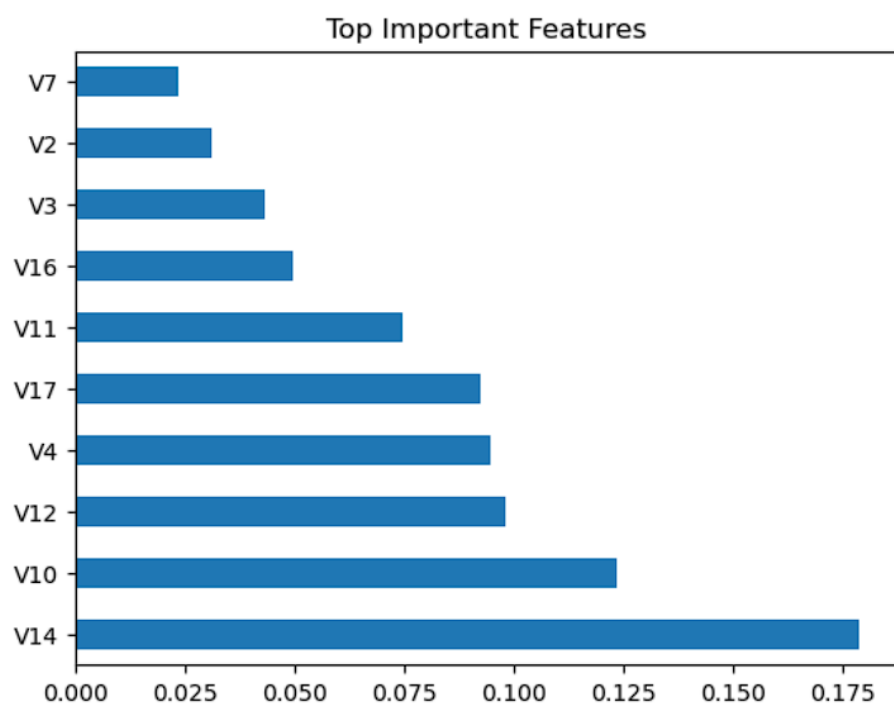
	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.70	0.82	0.75	98
accuracy			1.00	56962
macro avg	0.85	0.91	0.88	56962
weighted avg	1.00	1.00	1.00	56962

```
importance = rf_model.feature_importances_

feat_importance = pd.Series(importance, index=data.drop("Class",axis=1).columns)

feat_importance.nlargest(10).plot(kind='barh')

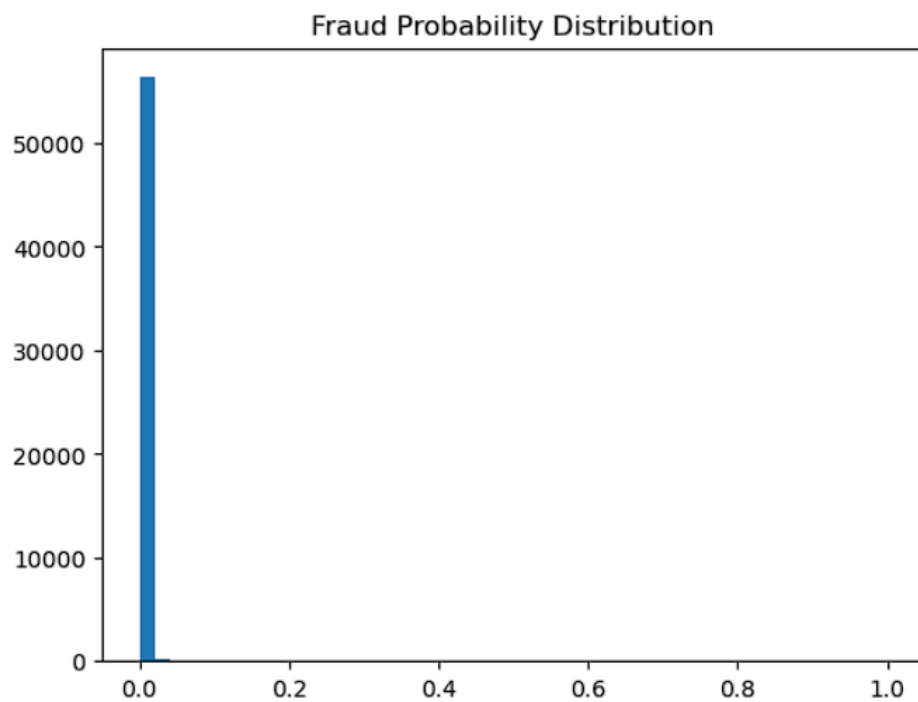
plt.title("Top Important Features")
plt.show()
```



Fraud Probability Visualization

```
fraud_prob = xgb_model.predict_proba(X_test)[:,-1]

plt.hist(fraud_prob, bins=50)
plt.title("Fraud Probability Distribution")
plt.show()
```



Save Models

```
joblib.dump(rf_model, "random_forest_model.pkl")  
joblib.dump(xgb_model, "xgboost_model.pkl")  
ann_model.save("ann_model.h5")
```

Predict New Transaction

```
def predict_transaction(transaction):  
    transaction = scaler.transform([transaction])  
    prediction = xgb_model.predict(transaction)  
  
    if prediction == 1:  
        return "Fraud Transaction"  
    else:  
        return "Legitimate Transaction"  
  
sample = X.iloc[0].values  
print(predict_transaction(sample))
```

Legitimate Transaction